w3 - Information Retrieval

Traditional Google Search cannot answer questions like “Do I need an umbrella today” because it requires contextual information that cannot be retrieved.

Information Retrieval

- **Document**: Unit of text indexed by system and retrieved.
- **Collection**: Set of documents used to satisfy user term requests
- **Term**: Word or phrase in a collection or query.
- **Query**: User information need expressed as a set of terms.

Inverted Index

Consists of dictionary and postings.

- Dictionary: List of terms in vocabulary. Accessed using B+ trees.
- Postings List: List of document IDs associated with each term, with term frequency (tf).

- Given a query, we use the inverted index to fetch all documents with at least one query term, called the **candidate documents.** Fetch together with information necessary to compute the tf-idf scores we need.

$$\text{Score}(q,d) = \sum_{t \in q \cap d} \frac{\text{tf}_{t,d} \cdot \text{idf}_t}{|d|}$$

Bag Of Words (BOW)

- A Vector representation doesn’t consider the ordering of the words, like clause ordering. The positional information of each term can be marked in the index to distinguish.
- Each document is a count vector in $|V|$ dimensions.

- Term frequency $tf_{t,d}$ is the number of times the term t occurs in document d. Term frequency is not useful enough, however; a term occurring 10x more is not 10x more relevant. Relevance does not increase proportionally with term frequency.
- Log Frequency Weighting (of term t in d):

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d}, & \text{if } \text{tf}_{t,d} > 0 \\ 0, & \text{otherwise} \end{cases}$$

  - Score for query-dcoument pair is the sum over terms in both q and d
  - $Score = \sum_{t \in q \cap d}(1 + log_{t,d})$. Note that it is the intersection, not union or separate count. Score is 0 if no common terms between query and document.

- Document Frequency
  - Document Frequency if the number of documents that contain t. It is inversely related to the informativeness ot t
  - Rare terms are more informative than frequent ones. A document containing a rare term is more likely to be relevant to a query containing that term. “Hadoop :)”
  - Lower the weights for frequent terms compared to rare terms.
  - Inverse Document Frequency is $log_{10}(N/df_t)$. Using log dampens the effect of idf
- Each term in the corpus has one df and one idf value.

- tf-idf
  - The product of df and idf for a term

$$\text{tf-idf}(t,d) = \log(1 + \text{tf}_{t,d}) \times \log_{10}(N / \text{df}_t)$$

  - Increases with number of occurrences within a document
  - Increase with rarity of term in collection of documents.

words \ docs	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	5.25	3.18	0	0	0	0.35
Brutus	1.21	6.1	0	1	0	0
Caesar	8.59	2.54	0	1.51	0.25	0
Calpurnia	0	1.54	0	0	0	0
Cleopatra	2.85	0	0	0	0	0
mercy	1.51	0	1.9	0.12	5.25	0.88
worser	1.37	0	0.11	4.15	0.25	1.95
...						

V

- There are many variants of tf-idf, can be computed with or without log, terms in query can be weighted as well, can divide by document length |d| etc.

Vector Space

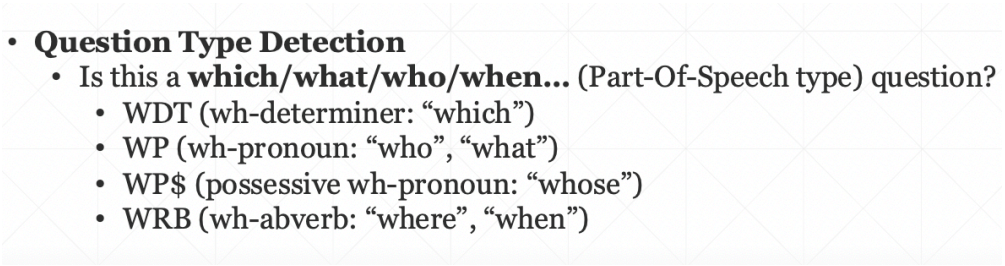
- We have a |V|-dimensional vector space. Each term in the vocabulary/corpus is an axis, and the documents are vectors in the space. The vector space is very high dimensional, and the documents are sparse vectors.

BERT

- Solves vocabular mismatch problem - where there is no exact overlap of words between document and query
- We use two separate encoder models, one to encode query, one to encode document. then use the dot product between the two vectors as the score.

Question Answering using IR

- Approach
  - Question processing - Detect question type, answer type, focus. Formulate queries to send to search engine.
  - Passage Retrieval - Retrieve and rank documents, break into suitable passages and re-rank
  - Answer processing - Extract candidate answers.
- Processing
  - We can use a taxonomy of classes to describe the entities and requirements of query (think the web of Description, Location, Abbreviation, Human, Numeric, Entity...)



  - Named entity type of answer
  - Formulate Query - choose query keywords for IR
- Answer Type Detection
  - Rule-based
    1. Regular expression based rules
    2. Other rules use question headword - first noun phrase after the wh-word.

- Machine Learning
    1. Treat as classification problem - Define taxonomy, annotate training data for each question type, train classifiers for each question type using set of features, inclusing handwritten rules.
    2. Features - Question words and phrases, parse features (headwords)
    3. Named entities e.g. PERSON, PLACE, ORGS

Common QA Evaluation Metrics

1. Accuracy (ratio of questions recalling good answers)
2. Mean Reciprocal Rank (MRR) - For each question return a ranked list of candidate answers. Query score is 1/Rank of first correct answer.
    a. If first answer correct: 1
    b. Second: 1/2
    c. Third: 1/3 etc
    d. Score is 0 if none of answers are correct

    Take mean over all N queries.